

# Scenario75 Cyber Range

---

## Technical Implementation Report

**Role:** Cybersecurity Engineer **Assessment:** Cyber Range Engineering (Red vs. Blue Lab) **Scenario:** Cookies Reuse & MFA Bypass **Technology:** Node.js, Express, Nginx, Docker Compose, Linux, OpenSSH **Repository:** <https://github.com/hangga/scenario75-cyber-range>

---

## 1. Executive Summary

This project implements a self-contained Red vs. Blue cyber range based on the provided Scenario75 assessment requirements.

The scenario simulates a web application containing an intentionally vulnerable authentication flow where a pre-MFA session cookie can be stolen through a reflected XSS vulnerability and subsequently replayed to obtain an authenticated administrator session without completing MFA.

The same environment also provides a Blue Team investigation path through deterministic Nginx and application logs. The logs contain sufficient evidence for an analyst to reconstruct the attack timeline, identify the attacker, correlate the XSS activity with the subsequent administrative access, identify the absence of an MFA verification request, and decode a Base64 forensic marker to obtain the Blue Team flag.

The implementation is designed to run inside an isolated Linux environment and can be deployed using Docker Compose on a single VM. The intended deployment model is a Proxmox VM, with the vulnerable application exposed through Nginx on TCP port 3075 and SSH access for the Blue Team on TCP port 2275.

The complete implementation, deployment scripts, deterministic forensic logs, verification script, and scenario documentation are available in the project repository.

---

## 2. Project Objectives

---

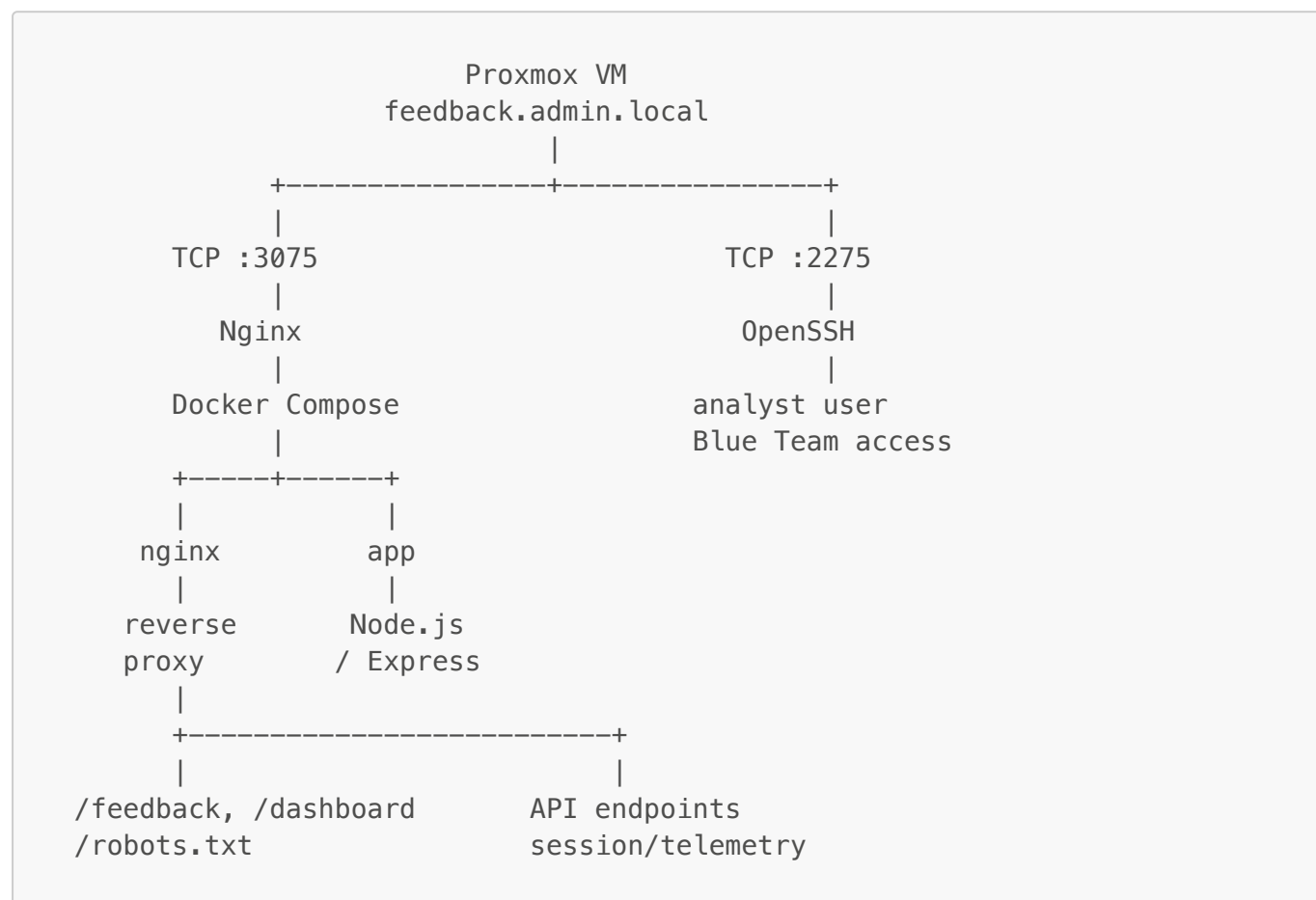
The primary objectives of the implementation are:

1. Build a reproducible Red vs. Blue cyber range.
2. Implement the required Cookies Reuse & MFA Bypass attack chain.
3. Provide a Node.js backend as required by the assessment.
4. Expose the web application through Nginx on TCP port 3075.
5. Provide Blue Team SSH access through TCP port 2275.
6. Store realistic access and application logs for forensic investigation.
7. Make the environment deployable on a single Linux VM.
8. Provide deterministic behavior so the scenario can be reset and repeated.
9. Provide an automated verification script for the important scenario requirements.
10. Keep the intentionally vulnerable components isolated for laboratory use.

### 3. Architecture

The cyber range uses a single Linux VM as the deployment boundary.

The VM contains Docker Compose services for the web application and Nginx, while OpenSSH provides the Blue Team access channel.



The application itself listens internally on port 3000, while Nginx provides the externally accessible HTTP interface on port 3075.

The lab hostname is:

```
feedback.admin.local
```

The Blue Team connects to the VM using:

```
ssh -p 2275 analyst@<VM_IP>
```

The supplied Blue Team credentials are:

```
Username: analyst  
Password: blue_team_rocks
```

Logs are stored under:

```
/opt/admin/logs/
```

with:

```
/opt/admin/logs/access.log  
/opt/admin/logs/error.log
```

---

## 4. Deployment Design

---

The environment is designed for a fresh Debian, Ubuntu, or Kali Linux VM.

The installation process is automated through scripts included in the repository.

The main deployment flow is:

```
sudo ./scripts/install_vm.sh  
sudo ./scripts/start_lab.sh
```

After deployment, the workstation resolves the internal lab hostname to the VM address:

```
<VM_IP> feedback.admin.local
```

The application can then be accessed through:

```
http://feedback.admin.local:3075/
```

The repository also provides a verification script:

```
./scripts/verify_lab.sh
```

The verifier checks externally observable requirements such as the HTTP port, response headers, robots.txt, cookie behavior, WAF responses, session replay behavior, flags, and forensic markers.

This provides a repeatable way to validate the range after deployment.

---

## 5. Red Team Scenario

---

### 5.1 Attack Objective

The Red Team objective is to bypass the intended MFA flow and obtain administrator access.

The attack chain is intentionally constructed around several weaknesses:

- Information disclosure through HTTP headers
- Endpoint discovery through robots.txt
- Client-accessible pre-MFA session cookie
- Reflected XSS
- Weak XSS filtering
- Weak cookie keyword filtering
- Replayable pre-MFA session state
- Missing server-side MFA enforcement during session replay

The complete attack chain is:

```
Recon
|
v
Hidden endpoint discovery
|
v
Pre-MFA cookie discovery
|
v
Reflected XSS
|
v
WAF bypass
|
v
Cookie theft
|
v
Cookie replay
|
v
Admin session
|
v
Dashboard access
```

```
|  
v  
RED FLAG
```

---

## 6. Red Team Phase 1: Reconnaissance

---

The first step is inspecting the HTTP response from the application.

The response exposes:

```
X-Powered-By: Node.js
```

This reveals the backend technology.

The corresponding flag is:

```
SCENARIO075{Node.js}
```

The `/robots.txt` endpoint contains a disallowed API endpoint:

```
/api/verify-mfa
```

This gives the Red Team an important clue about the authentication architecture.

The corresponding flag is:

```
SCENARIO075{/api/verify-mfa}
```

The `/dashboard` endpoint is also an important discovery because it represents the protected administrative area.

The corresponding flag is:

```
SCENARIO075{/dashboard}
```

The application's HTML also contains an ASCII-art comment that hints toward `robots.txt`.

This provides another discovery clue:

```
SCENARIO075{robots.txt}
```

---

## 7. Red Team Phase 2: Pre-MFA Session and XSS

---

The application issues an unauthenticated cookie:

```
pre_mfa_session=pending_mfa_verification
```

The cookie consists of:

```
Cookie name:  
pre_mfa_session  
  
Cookie value:  
pending_mfa_verification
```

The corresponding flags are:

```
SCENARIO075{pre_mfa_session}  
SCENARIO075{pending_mfa_verification}
```

An important security property of the scenario is that this cookie is not protected with the **HttpOnly** attribute.

Therefore, JavaScript executing in the browser can access it.

The application accepts feedback through:

```
POST /feedback
```

A standard **<script>** payload is intentionally blocked by the WAF and returns:

```
403
```

This demonstrates the first filtering mechanism.

The corresponding flag is:

```
SCENARIO75{403}
```

The filtering is intentionally weak. An SVG event handler can bypass the simplistic filter:

```
<svg onload=...>
```

The corresponding flag is:

```
SCENARIO75{<svg>}
```

The WAF also attempts to block direct access to the cookie through the `document.cookie` keyword.

The scenario therefore requires the Red Team to use bracket notation and string concatenation:

```
window['docu'+ 'ment'] ['coo'+ 'kie']
```

The corresponding flag is:

```
SCENARIO75{window['docu'+ 'ment'] ['coo'+ 'kie']}
```

The XSS payload then sends the stolen cookie to the laboratory telemetry endpoint.

The application allows the `fetch` API for this purpose.

The corresponding flag is:

```
SCENARIO75{fetch}
```

The important security weakness here is not simply XSS itself. The vulnerability becomes significantly more serious because the pre-MFA session cookie is accessible to JavaScript and remains useful for authentication-state escalation.

---

## 8. Red Team Phase 3: Cookie Replay and MFA Bypass

---

The stolen pre-MFA cookie can be replayed against:

```
/api/session/replay
```

The important design flaw is that the server accepts the valid pre-MFA session state and creates an administrative session without requiring a successful request to:

```
/api/verify-mfa
```

The resulting administrator session uses the:

```
adm_sess
```

prefix.

The corresponding flag is:

```
SCENARIO075{adm_sess}
```

The absence of an MFA verification request is a central part of the scenario.

The corresponding flag is:

```
SCENARIO075{/api/verify-mfa}
```

At this stage, the Red Team can access:

```
/dashboard
```

using the replayed administrator session.

The dashboard reflects the attacker-controlled payload inside a CSS class:

```
.xss-payload
```

The corresponding flag is:

```
SCENARIO075{xss-payload}
```

The final Red Team flag is:

```
SCENARIO075{RED_C00k13_MFA_Byp4ss_0wn3d}
```

---

## 9. Blue Team Investigation

---

The Blue Team investigation is designed to demonstrate that successful exploitation should leave enough telemetry to reconstruct the attack.

The Blue Team first connects through SSH:

```
ssh -p 2275 analyst@<VM_IP>
```

The relevant evidence is located under:

```
/opt/admin/logs/
```

The two primary files are:

```
access.log  
error.log
```

The investigation focuses on:

- Source IP
- User-Agent
- Request sequence
- WAF blocks
- Successful dashboard access
- Authentication-related anomalies
- Cookie reuse
- Missing MFA verification traffic
- Suspicious X-Forwarded-For data

---

## 10. Identifying the Attacker

---

The logs identify the attacker IP as:

```
10.10.14.50
```

The corresponding flag is:

```
SCENARIO075{10.10.14.50}
```

The attacker belongs to:

```
10.10.14.0/24
```

The corresponding flag is:

```
SCENARIO075{10.10.14.0/24}
```

The attacker User-Agent is:

```
Mozilla/5.0
```

The corresponding flag is:

```
SCENARIO075{Mozilla/5.0}
```

The logs also contain legitimate administrative background traffic from:

```
192.168.1.100
```

This provides a useful comparison between expected and suspicious traffic.

---

## 11. Reconstructing the Attack Timeline

---

The deterministic forensic logs allow the Blue Team to establish the sequence of events.

The first WAF block occurs at:

```
18:50:15
```

This corresponds to the blocked `<script>` payload.

The corresponding flags are:

```
SCENARIO075{18:50:15}  
SCENARIO075{403}
```

The attacker subsequently obtains successful administrative access.

The `/dashboard` request returns:

```
200
```

at:

```
18:51:55
```

The corresponding flags are:

```
SCENARIO075{200}  
SCENARIO075{18:51:55}
```

The important forensic observation is that the attacker does not make a request to:

```
/api/verify-mfa
```

The corresponding Blue Team finding is:

```
SCENARIO075{No}
```

This absence is significant because the attacker reaches an authenticated administrative endpoint without generating the expected MFA verification request.

Later, the application records an authentication anomaly at:

```
18:53:10
```

with the exact message:

```
Authentication bypass anomaly
```

The corresponding flag is:

```
SCENARIO075{Authentication bypass anomaly}
```

Cookie reuse events are classified as:

```
CRITICAL
```

The corresponding flag is:

```
SCENARIO075{CRITICAL}
```

---

## 12. Base64 Forensic Marker

---

The Blue Team investigation also contains a suspicious `X-Forwarded-For` value.

The value is:

```
UEhBTlRPTUdSSUR7QkxVRV9MMGdfSHVudDNyX000c3Qzcn0}
```

The scenario requires the analyst to recognize that this value is Base64 encoded.

The corresponding flag is:

```
SCENARIO075{Base64}
```

The encoded value is intentionally fixed-length.

The corresponding flag is:

```
SCENARIO075{44}
```

The value can be investigated using:

```
printf '%s' 'UEhBTlRPTUdSSUR7QkxVRV9MMGdfSHVudDNyX000c3Qzcn0}' | base64 -d
```

The resulting forensic marker leads to the Blue Team flag:

```
SCENARIO075{BLUE_L0G_HUnt3r_M4st3r}
```

The Blue Team investigation chain is therefore:

```
Logs
|
v
Suspicious IP
|
v
XSS attempt
|
v
Successful dashboard access
|
v
No MFA verification request
|
v
Cookie reuse
|
v
Authentication anomaly
|
v
Base64 forensic marker
|
v
BLUE FLAG
```

---

## 13. Logging and Telemetry Design

---

The lab intentionally separates access-level and application-level evidence.

## access.log

The Nginx-style access log provides information such as:

- Source IP
- Request method
- Request path
- HTTP status
- User-Agent
- Request timing
- Forwarded IP information

This allows the Blue Team to reconstruct the network-facing portion of the attack.

## error.log

The application error log provides higher-level security events such as:

- WAF blocks
- Cookie reuse
- Authentication anomalies
- Suspicious application behavior

The combination of both logs allows the analyst to correlate HTTP activity with application-level security events.

The logs are deterministic so that the Blue Team exercise can be repeated consistently.

---

# 14. Intentional Vulnerabilities

---

The vulnerabilities in this project are intentional and exist to support the assessment scenario.

The main weaknesses are:

### 14.1 Information Disclosure

The application exposes its backend technology through:

```
X-Powered-By: Node.js
```

### 14.2 Predictable Pre-MFA State

The application provides a pre-MFA session state that can be replayed.

### 14.3 Missing HttpOnly Protection

The `pre_mfa_session` cookie is accessible through JavaScript.

## 14.4 Reflected XSS

User-controlled feedback content can be reflected into the application.

## 14.5 Weak WAF

The WAF relies on simplistic pattern matching and can be bypassed using alternative HTML event syntax and JavaScript expression construction.

## 14.6 Broken MFA Enforcement

The session replay endpoint accepts the pre-MFA session and creates an administrator session without requiring successful MFA verification.

## 14.7 Insufficient Session Binding

A stolen pre-MFA session token can be replayed independently of the original browser context.

These weaknesses are intentionally implemented to create a complete attack chain rather than isolated vulnerability demonstrations.

---

# 15. Automated Verification

---

The project includes:

```
scripts/verify_lab.sh
```

The verification script is intended to validate the important externally observable behavior of the scenario.

The checks include:

- Application availability
- Port configuration
- HTTP response headers
- robots.txt
- Initial cookie
- WAF behavior
- XSS behavior
- Session replay
- Required flags
- Forensic log markers

This reduces the risk of accidental scenario regressions when the environment is rebuilt.

For a cyber range, deterministic verification is particularly useful because changes to the application can otherwise silently break one of the required Red or Blue objectives.

---

## 16. Reset and Reproducibility

---

The scenario is intentionally deterministic.

The forensic logs can be regenerated using:

```
sudo ./scripts/generate_logs.sh
```

The application can be restarted using:

```
sudo docker compose -f /opt/scenario75/docker-compose.yml down
sudo docker compose -f /opt/scenario75/docker-compose.yml up -d --build
```

This allows the same scenario to be executed multiple times during testing, demonstration, or evaluation.

---

## 17. Requirement Coverage

---

| Assessment Requirement | Implementation                                          |
|------------------------|---------------------------------------------------------|
| Linux environment      | Designed for Debian, Ubuntu, or Kali VM                 |
| Docker                 | Docker Compose deployment                               |
| Single VM              | Complete lab can run inside one VM                      |
| Proxmox compatible     | VM-based deployment model                               |
| Node.js backend        | Node.js / Express application                           |
| HTTP port 3075         | Nginx exposed on TCP 3075                               |
| Internal hostname      | feedback.admin.local                                    |
| SSH port 2275          | OpenSSH configured for Blue Team access                 |
| Blue Team user         | analyst                                                 |
| Red reconnaissance     | Headers, robots.txt, HTML hints, cookie discovery       |
| XSS challenge          | /feedback                                               |
| WAF challenge          | <script> blocked and SVG bypass                         |
| Cookie theft           | JavaScript-accessible pre-MFA cookie                    |
| Cookie replay          | /api/session/replay                                     |
| MFA bypass             | Administrative session created without MFA verification |

| Assessment Requirement  | Implementation                                        |
|-------------------------|-------------------------------------------------------|
| Admin endpoint          | <code>/dashboard</code>                               |
| Red flag                | <code>SCENARIO075{RED_C00k13_MFA_Byp4ss_0wn3d}</code> |
| Blue logs               | <code>/opt/admin/logs</code>                          |
| Attacker identification | IP and User-Agent                                     |
| Timeline analysis       | Deterministic timestamps                              |
| MFA absence detection   | No attacker request to <code>/api/verify-mfa</code>   |
| Cookie reuse detection  | CRITICAL event                                        |
| Authentication anomaly  | Application error log                                 |
| Base64 investigation    | X-Forwarded-For forensic marker                       |
| Blue flag               | <code>SCENARIO075{BLUE_L0G_HUnt3r_M4st3r}</code>      |
| Automated validation    | <code>scripts/verify_lab.sh</code>                    |
| Reset capability        | <code>scripts/generate_logs.sh</code>                 |

## 18. Repository Structure

The repository is organized around the major components of the cyber range:

```
scenario75-cyber-range/  
├── app/  
├── docs/  
├── logs/  
├── nginx/  
├── scripts/  
├── docker-compose.yml  
├── README.md  
├── SCENARIO0.md  
└── package-lock.json
```

The separation keeps the vulnerable application, reverse proxy configuration, deployment automation, forensic data, and scenario documentation independently maintainable.

The repository is publicly available at:

<https://github.com/hangga/scenario75-cyber-range>

## 19. Security Considerations

This project intentionally contains exploitable vulnerabilities and should not be deployed on a production network or exposed to the public Internet.

The environment is intended for:

- Cybersecurity training
- Red Team exercises
- Blue Team investigation exercises
- Cyber range demonstrations
- Security engineering assessment
- Controlled vulnerability research

The vulnerable application should remain isolated from production systems.

The deterministic credentials, cookies, flags, and logs are part of the exercise design and should not be reused in real systems.

---

## 20. Conclusion

---

Scenario75 demonstrates the complete lifecycle of a controlled web security exercise, from reconnaissance and exploitation to forensic investigation.

From the Red Team perspective, the scenario demonstrates how several individually simple weaknesses can be chained together:

```
Information Disclosure
      +
Weak XSS Filtering
      +
Reflected XSS
      +
Client-Accessible Session Cookie
      +
Replayable Pre-MFA State
      +
Missing MFA Enforcement
      =
Administrative Access
```

From the Blue Team perspective, the same attack produces a recognizable sequence of forensic indicators:

```
WAF Block
  ->
Suspicious Source
  ->
Successful Dashboard Access
```

```
->  
No MFA Verification  
->  
Cookie Reuse  
->  
Authentication Anomaly  
->  
Forensic Base64 Marker
```

The resulting cyber range is self-contained, reproducible, and verifiable. It provides both an offensive attack path and a defensive investigation path within the same isolated environment.

The implementation is available in the accompanying Git repository and includes deployment automation, Docker Compose configuration, Node.js application code, Nginx configuration, deterministic forensic logs, scenario documentation, and automated verification.